

Git Avanzado

Arturo Silvelo

Try New Roads

Configuración avanzada de Git

Configuración avanzada de Git

Git no solo guarda tu código, también registra información sobre **quién hizo qué cambios y cómo se gestionan los repositorios**.

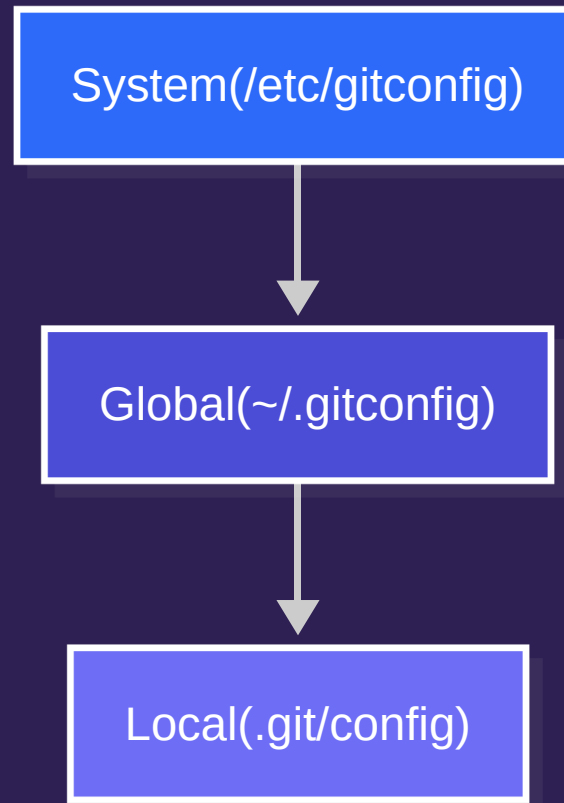
- Tu identidad (nombre y correo para los commits)
- Alias para comandos frecuentes
- Comportamiento de fusión y merges
- Optimización y limpieza de repositorios
- Reglas de seguridad y políticas de la organización

Por qué es importante:

- Evita errores de identificación de commits.
- Mejora la productividad con comandos abreviados.
- Facilita la colaboración en equipos grandes.
- Mantiene los repositorios eficientes y limpios.

Niveles de configuración en Git

Git tiene **tres niveles** de configuración:



1. System (sistema)

- Aplica a **todos los usuarios y repositorios** en el equipo.
- Ubicación: `/etc/gitconfig` o `C:\ProgramData\Git\config`.
- Ideal para establecer reglas globales de la organización.

Configuración	Comando	Explicación
Editor por defecto	<code>git config --system core.editor "nano"</code>	Todos los usuarios usarán <code>nano</code> al abrir editores de commits o merges.
Colores de la terminal	<code>git config --system color.ui auto</code>	Activa colores por defecto para todos los usuarios y repositorios.
Política de line endings	<code>git config --system core.autocrlf true</code>	Evita problemas de saltos de línea en equipos Windows/Linux.
Path de hooks globales	<code>git config --system core.hooksPath /opt/git/hooks</code>	Define hooks que se aplican a todos los repositorios del equipo.

2. Global (usuario)

- Aplica a todos los repositorios de un usuario.
- Ubicación: `~/.gitconfig`.
- Se configura identidad, alias y preferencias personales.

Configuración	Comando	Explicación
Nombre de usuario	<pre>git config --global user.name "Arturo Silvelo"</pre>	Se usa en todos los commits del usuario.
Correo de usuario	<pre>git config --global user.email "arturo@example.com"</pre>	Asociado a todos los commits del usuario.
Alias de comandos	<pre>git config --global alias.co checkout</pre>	Facilita comandos frecuentes.
Fusión automática con rebase	<pre>git config --global pull.rebase true</pre>	El usuario siempre hará rebase al hacer pull, para un historial lineal.
Editor personalizado	<pre>git config --global core.editor "code --wait"</pre>	Configura el editor que prefiera el usuario, sin afectar a otros.

3. Local (repositorio)

- Aplica solo al repositorio actual.
- Ubicación: `.git/config`.
- Útil para proyectos específicos o clientes distintos.

Configuración	Comando	Explicación
Nombre de equipo o proyecto	<code>git config --local user.name "Equipo ProyectoX"</code>	Sobrescribe la identidad global solo para este proyecto.
Ignorar reglas específicas	<code>git config --local core.excludesFile .gitignore_global</code>	Define reglas de exclusión solo para este repositorio.
Branch principal	<code>git config --local init.defaultBranch main</code>	Configura la rama por defecto para este repositorio.
Hooks específicos	<code>git config --local core.hooksPath .githooks</code>	Usa hooks que solo se aplican a este repositorio.
Estrategia de merges	<code>git config --local merge.ff false</code>	Fuerza merges con commit, sin fast-forward, solo en este repositorio.

Alias

Un **alias** en Git es una forma de crear comandos abreviados o personalizados para tareas frecuentes.

- Permite ahorrar tiempo y evitar errores de tipeo.
- Se definen en la configuración global o local.
- Pueden ser simples (un comando) o complejos (comandos encadenados).

Ejemplo de alias avanzado

Puedes crear alias que incluyan opciones o comandos complejos:

```
git config --global alias.lg "log --oneline --decorate --graph --all"
```

Ahora puedes ejecutar:

```
git lg
```

para ver un historial visual de todas las ramas.

Gestión de credenciales y seguridad

En Git y GitHub, la **autenticación y las credenciales** son esenciales para controlar quién puede leer o modificar un repositorio.

Una mala configuración puede provocar:

- Push sin permisos
- Exposición de datos sensibles
- Confusión entre identidades si se usan varias cuentas

Métodos de autenticación en GitHub

1. SSH Keys (recomendado para desarrolladores)

- Clave pública y privada.
- Conexión segura sin pedir contraseña en cada push/pull.

Configuración de SSH

- **Generar clave SSH:**

```
ssh-keygen -t ed25519 -C "tu_email@example.com"
```

- Copiar contenido de ~/.ssh/id_ed25519.pub
- Añadirlo en Settings → SSH and GPG keys.

2. Personal Access Tokens (PAT)

- Se usan en lugar de la contraseña para HTTPS.
- Se pueden configurar con permisos específicos (repo, workflow, packages, etc.).

Configuración de Personal Access Token (PAT)

1. Crear token en Settings → Developer settings → Personal access tokens → Tokens (classic).
2. Elegir permisos mínimos necesarios: repo para repositorios privados, workflow para GitHub Actions, etc.

Manejo de múltiples cuentas

- Problema: un mismo equipo puede necesitar acceso a repositorios de cuentas distintas.
- Solución: usar configuración SSH con archivos de config:

```
# ~/.ssh/config
Host github-personal
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_personal

Host github-work
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_work
```

- Clonar usando el alias definido

```
git clone git@github-personal:usuario/repositorio.git
git clone git@github-work:empresa/repositorio.git
```

Protección de secretos

Evitar subir información sensible al repositorio y proteger credenciales.

- Subir secretos (contraseñas, claves, tokens) puede comprometer proyectos y cuentas.
- Los errores son fáciles de cometer: accidentalmente un `.env` o un `credentials.json`.
- Mantener un repositorio seguro evita fugas de datos y problemas legales o empresariales.

Estrategias de protección

```
git config --global core.excludesFile ~/.gitignore_global
```

```
# .gitignore_global
```

```
_.env
```

```
_.key
```

```
_.pem
```

```
_.crt
```

```
credentials.json
```

Internals de Git

Internals de Git

- Git es un sistema **basado en snapshots**, no en diferencias entre archivos.
- Cada commit es un **objeto que apunta a un árbol de archivos**, que a su vez apunta a blobs (contenido de archivos).
- Git forma un **DAG (Directed Acyclic Graph)** de commits, lo que permite:
 - Historial lineal o ramificado
 - Fusión de ramas eficiente
 - Recuperación de cambios incluso eliminados

Estructura interna de Git

- Cada repositorio Git tiene un directorio oculto `.git` en la raíz.
- Contiene toda la información necesaria para versionar el proyecto: commits, ramas, configuraciones, hooks, etc.
- Si borras `.git`, pierdes el historial, pero los archivos de trabajo permanecen.

Elemento	Función
HEAD	activa
config	Configuración local del repositorio
hooks/	Scripts que se ejecutan en eventos de Git (pre-commit, pre-push, etc.)
objects/	Contiene todos los objetos de Git (blobs, trees, commits, tags)
refs/	Contiene referencias a ramas (heads) y tags (tags)
logs/	Reflog: historial de movimientos de HEAD y ramas
info/	Archivos de configuración interna, como .exclude

Objects

- Cada commit es un objeto en `.git/objects` con un SHA único.
- Los almacena de manera comprimida y codificada
- Contiene:
 - Referencia al árbol
 - Commit padre
 - Autor y fecha
 - Mensaje

Tipos de Objetos

- Blob → contenido de archivo
- Tree → lista de archivos y subdirectorios, apunta a blobs y otros trees
- Commit → apunta a un tree y a sus padres

- Contenido de un commit

```
git cat-file -p 7ca1c8b30d0cbcf0d91edb2f379d6b7afa3eb2cc
tree 56c522739908ff6c35bd85bc9944bb8585ab8bef
author Arturo Silvelo <asilvelo@trynewroads.com> 1760604329 +0200
committer Arturo Silvelo <asilvelo@trynewroads.com> 1760604329 +0200

Commit A: archivo1.txt
```

- Contenido de un tree

```
git cat-file -p 56c522739908ff6c35bd85bc9944bb8585ab8bef
100644 blob 7d249aa07f90adbb03a9084d962387de486f90b1      archivo1.txt
```

- Contenido de un blob

```
git cat-file -p 7d249aa07f90adbb03a9084d962387de486f90b1
Primer archivo
```

Ramas y referencias

Las ramas son simplemente archivos de texto en `.git/refs/heads/` que contienen el SHA del último commit de esa rama.

```
ls .git/refs/heads/  
feature  main
```

```
cat .git/refs/heads/feature  
a2c810f361157b4b793790c28c2e6e90b47b03a4
```

Tags funcionan de manera similar en `.git/refs/tags/`

```
cat .git/refs/tags/v0.1  
4e665c4ebd876630946055258c84816cd71a4b32
```

Logs

La carpeta `.git/logs/` contiene los registros de movimientos de las referencias en tu repositorio, principalmente el reflog.

Historial completo de todos los movimientos de HEAD

```
$cat .git/logs/HEAD
# ó
$git reflog
5d54f8c (HEAD -> main) HEAD@{0}: checkout: moving from rama to main
bc69376 (tag: v0.1) HEAD@{1}: checkout: moving from main to rama
5d54f8c (HEAD -> main) HEAD@{2}: commit: Commit D: cambio en main
bc69376 (tag: v0.1) HEAD@{3}: checkout: moving from feature to main
a2c810f (feature) HEAD@{4}: commit: Commit C: feature.txt
bc69376 (tag: v0.1) HEAD@{5}: checkout: moving from main to feature
bc69376 (tag: v0.1) HEAD@{6}: commit: Commit B: archivo2.txt
7ca1c8b HEAD@{7}: commit (initial): Commit A: archivo1.txt
```


config

Configuración local del repositorio

```
git config --local user.name Teacher
```

```
cat .git/config  
[user]  
    name = Teacher
```

Archivos temporales

- **COMMIT_EDITMSG:**

Guarda el mensaje del commit actual mientras se edita.
Útil para hooks o para recuperar el mensaje si falla el commit.

```
cat .git/COMMIT_EDITMSG
```

- **MERGE_MSG:**

Se crea durante un merge para guardar el mensaje por defecto del merge.

Stash

stash

Es una funcionalidad de Git que permite guardar temporalmente los cambios no confirmados (tanto en el área de trabajo como en el stage) sin necesidad de hacer un commit.

- Es útil cuando necesitas cambiar de rama o actualizar tu repositorio, pero no quieres perder ni comprometer tus cambios actuales.
- Los cambios guardados en el stash se almacenan como objetos especiales en `.git/refs/stash` y pueden recuperarse más tarde.

Para guardar tus cambios actuales en el stash, simplemente ejecuta:

```
git stash
```

```
cat .git/refs/stash  
2b066d788c3eadc50742f07bf0e25a068f92547b
```

Esto:

- Guarda todos los cambios no confirmados y limpia tu directorio de trabajo.
- Puedes continuar trabajando, cambiar de rama o actualizar tu repo sin perder esos cambios.

Trabajar con stash

- Listar

```
git stash list
stash@{0}: WIP on main: ce455ac Merge branch 'feature'
```

- Crear con mensaje

```
git stash save "Mensaje"
```

- Archivo específico

```
git stash push archivo.txt
```

- Recuperar

```
git stash apply # Último
git stash apply stash@{0} # Especifico
```

- Recuperar y eliminar

```
git stash pop
```

- Eliminar

```
git stash clear # Todos
git stash drop stash@{0} # Especifico
```

- Ver el contenido

```
git stash show -p stash@{0}
```

Reflog

Reflog

Es un registro interno donde Git guarda todos los movimientos recientes de las referencias importantes, especialmente `HEAD` y las ramas.

- Permite rastrear y recuperar cambios recientes, incluso si se han perdido ramas o commits por error.
- Es fundamental para la recuperación de trabajo perdido tras un reset, rebase, merge o eliminación accidental.

¿Para qué sirve el reflog?

- **Recuperar commits o ramas borradas accidentalmente.**
- Volver a un estado anterior tras un reset, rebase o merge problemático.
- Auditar el historial de cambios recientes, incluso si no aparecen en `git log`.

Ejemplo práctico: Recuperar un commit perdido

- Crear nueva rama

```
git switch -c feature-delete
```

- Crear varios commits

```
touch f.txt  
git add .  
git commit -m 'Commit F: f.txt'  
echo "Hola mundo" > f.txt  
git commit -am "Commit F: add content"
```

- Borrar rama

```
git switch main  
git branch -D feature-delete
```

- Buscar los reflog

```
git reflog
```

- Recuperar la rama

```
git checkout -b rama-recuperada HEAD@{1}
```

- Comprobar

```
git switch rama-recuperada  
git log
```

Gestión y limpieza del reflog

- El reflog se limpia automáticamente tras un periodo (por defecto, 90 días).
- Puedes forzar la limpieza con:

```
git reflog expire --expire=now --all  
git gc --prune=now
```